Bayesian machine learning

Created by: Roger Grosse
Intended for: beginning machine learning researchers, practitioners

Bayesian statistics is a branch of statistics where quantities of interest (such as parameters of a statistical model) are treated as random variables, and one draws conclusions by analyzing the posterior distribution over these quantities given the observed data. While the core ideas are decades or even centuries old, Bayesian ideas have had a big impact in machine learning in the past 20 years or so because of the flexibility they provide in building structured models of real world phenomena. Algorithmic advances and increasing computational resources have made it possible to fit rich, highly structured models which were previously considered intractable.

This roadmap is meant to give pointers to a lot of the key ideas in Bayesian machine learning. If you're considering applying Bayesian techniques to some problem, you should learn everything in the "core topics" section. Even if you just want to use a software package such as BUGS , Infer.NET , or Stan , this background will help you figure out the right questions to ask. Also, if the software doesn't immediately solve your problem, you'll need to have a rough mental model of the underlying algorithms in order to figure out why.

If you're considering doing research in Bayesian machine learning, the core topics and many of the advanced topics are part of the background you're assumed to have, and the papers won't necessarily provide citations. There's no need to go through everything here in linear

order (the whole point of Metacademy is to prevent that!), but hopefully this roadmap will help you learn these things as you need them. If you wind up doing research in Bayesian machine learning, you'll probably wind up learning all of these topics at some point.

Core topics

This section covers the core concepts in Bayesian machine learning. If you want to use this set of tools, I think it's worth learning everything in this section.

Central problems

What is Bayesian machine learning? Generally, Bayesian methods are trying to solve one of the following problems:

+ parameter estimation. Suppose you have a statistical model of some domain, and you want to use it to make predictions. Or maybe you think the parameters of the model are meaningful, and you want to fit them in order to learn something about the world. The Bayesian approach is to compute or approximate the posterior distribution over the parameters given the observed data.
    - Often, you want to use the model to choose actions. Bayesian decision theory is a framework for doing this.
+ model comparison: You may have several different models under consideration, and you want to know which is the best match to your data. A common case is that you have several models of the same form of differing complexities, and you want to trade off the complexity with the degree of fit.
    - Rather than choosing a single model, you can define a prior over the models themselves, and average the predictions with respect to the posterior over models. This is known as Bayesian model averaging.

It's also worth learning the basics of Bayesian networks (Bayes nets), since the notation is used frequently when talking about Bayesian models. Also, because Bayesian methods treat the model parameters as random variables, we can represent the Bayesian inference problems themselves as Bayes nets!

The readings for this section will tell you enough to understand *what problems Bayesian methods are meant to address*, but won't tell you how to actually solve them in general. That is what the rest of this roadmap is for.

Non-Bayesian techniques

As background, it's useful to understand how to fit generative models in a non-Bayesian way. One reason is that these techniques can be considerably simpler to implement, and often they're good enough for your goals. Also, the Bayesian techniques bear close similarities to these, so they're often helpful analogues for reasoning about Bayesian techniques.

Most basically, you should understand the notion of generalization, or how well a machine learning algorithm performs on data it hasn't seen before. This is fundamental to evaluating any sort of machine learning algorithm. You should also understand the following techniques:

+ maximum likelihood, a criterion for fitting the parameters of a generative model
+ regularization, a method for preventing overfitting
+ the EM algorithm, an algorithm for fitting generative models where each data point has associated latent (or unobserved) variables

Basic inference algorithms

In general, Bayesian inference requires answering questions about the posterior distribution

over a model's parameters (and possibly latent variables) given the observed data. For some simple models, these questions can be answered analytically. However, most of the time, there is no analytic solution, and we need to compute the answers approximately.

If you need to implement your own Bayesian inference algorithm, the following are probably the simplest options:

+ MAP estimation, where you approximate the posterior with a point estimate on the optimal parameters. This replaces an integration problem with an optimization problem. This doesn't mean the problem is easy, since the optimization problem is often itself intractable. However, it often simplifies things, because software packages for optimization tend to be more general and robust than software packages for sampling.
+ Gibbs sampling, an iterative procedure where each random variable is sampled from its conditional distribution given the remaining ones. The result is (hopefully) an approximate sample from the posterior distribution.

You should also understand the following general classes of techniques, which include the majority of the Bayesian inference algorithms used in practice. Their general formulations are too generic to be relied on most of the time, but there are a lot of special cases which are very powerful:

+ Markov chain Monte Carlo, a general class of sampling-based algorithms based on running Markov chains over the parameters whose stationary distribution is the posterior distribution.
    - In particular, Metropolis-Hastings (M-H) is a recipe for constructing valid MCMC chains. Most practical MCMC algorithms, including Gibbs sampling, are special cases of M-H.
+ Variational inference, a class of techniques which try to approximate the intractable posterior distribution with a tractable distribution. Generally, the parameters of the

tractable approximation are chosen to minimize some measure of its distance from the true posterior.

Models

The following are some simple examples of generative models to which Bayesian techniques are often applied.

+ mixture of Gaussians, a model where each data point belongs to one of several "clusters," or groups, and the data points within each cluster are Gaussian distributed. Fitting this model often lets you infer a meaningful grouping of the data points.
+ factor analysis, a model where each data point is approximated as a linear function of a lower dimensional representation. The idea is that each dimension of the latent space corresponds to a meaningful factor, or dimension of variation, in the data.
+ hidden Markov models, a model for time series data, where there is a latent discrete state which evolves over time.

While Bayesian techniques are most closely associated with generative models, it's also possible to apply them in a discriminative setting, where we try to directly model the conditional distribution of the targets given the observations. The canonical example of this is Bayesian linear regression.

Bayesian model comparison

The section on inference algorithms gave you tools for approximating posterior inference. What about model comparison? Unfortunately, most of the algorithms are fairly involved, and you probably don't want to implement them yourself until you're comfortable with the advanced inference algorithms described below. However, there are two fairly crude approximations which are simple to implement:

+ the Bayesian information criterion (BIC), which simply takes the value of the MAP solution and adds a penalty proportional to the number of parameters
+ the Laplace approximation, which approximates the posterior distribution with a Gaussian centered around the MAP estimate.

Advanced topics

This section covers more advanced topics in Bayesian machine learning. You can learn about the topics here in any order.

Models

The "core topics" section listed a few commonly used generative models. Most datasets don't fit those structures exactly, however. The power of Bayesian modeling comes from the flexibility it provides to build models for many different kinds of data. Here are some more models, in no particular order.

+ logistic regression, a discriminative model for predicting binary targets given input features
+ Bayesian networks (Bayes nets). Roughly speaking, Bayes nets are directed graphs which encode patterns of probabilistic dependencies between different random variables, and are typically chosen to represent the causal relationships between the variables. While Bayes nets can be learned in a non-Bayesian way, Bayesian techniques can be used to learn both the parameters and structure (the set of edges) of the network.
    - Linear-Gaussian models are an important special case where the variables of the network are all jointly Gaussian. Inference in these networks is often tractable even in cases where it's intractable for discrete networks with the same structure.

+ latent Dirichlet allocation, a "topic model," where a set of documents (e.g. web pages) are each assumed to be composed of some number of topics, such as computers or sports. Related models include nonnegative matrix factorization and probabilistic latent semantic analysis.
+ linear dynamical systems, a time series model where a low-dimensional gaussian latent state evolves over time, and the observations are noisy linear functions of the latent states. This can be thought of as a continuous version of the HMM. Inference in this model can be performed exactly using the Kalman filter and smoother.
+ sparse coding, a model where each data point is modeled as a linear combination of a small number of elements drawn from a larger dictionary. When applied to natural image patches, the learned dictionary resembles the receptive fields of neurons in the primary visual cortex. See also a closely related model called independent component analysis.

Bayesian nonparametrics

All of the models described above are *parametric*, in that they are represented in terms of a fixed, finite number of parameters. This is problematic, since it means one needs to choose a parameter for, e.g., the number of clusters, and this is rarely known in advance.

This problem may not seem so bad for the models described above, because for simple models such as clustering, one can typically choose good parameters using cross-validation. However, many widely used models are far more complex, involving many independent clustering problems, where the numbers of clusters can vary from a handful to thousands.

Bayesian nonparametrics is an ongoing research area within machine learning and statistics which sidesteps this problem by defining models which are *infinitely complex*. We cannot explicitly represent infinite objects in their entirety, of course, but the key insight is that for a

finite dataset, we can still perform posterior inference in the models while only explicitly representing a finite portion of them.

Here are some of the most important building blocks which are used to construct Bayesian nonparametric models:

+ Gaussian processes are priors over functions such that the values sampled at any finite set of points are jointly Gaussian. In many cases, posterior inference is tractable. This is probably the default thing to use if you want to put a prior over functions.
+ the Chinese restaurant process, which is a prior over partitions of an infinite set of objects.
  - This is most commonly used in clustering models when one doesn't want to specify the number of components in advance. The inference algorithms are fairly simple and well understood, so there's no reason not to use a CRP model in place of a finite clustering model.
  - This process can equivalently be viewed as Dirichlet process.
+ the hierarchical Dirichlet process, which involves a set of Dirichlet processes which share the same base measure, and the base measure is itself drawn from a Dirichlet process.
+ the Indian buffet process, a prior over infinite binary matrices such that each row of the matrix has only a finite number of 1's. This is most commonly used in models where each object can have various attributes. I.e., rows of the matrix correspond to objects, columns correspond to attributes, and an entry is 1 if the object has the attribute.
  - The simplest example is probably the IBP linear-Gaussian model, where the observed data are linear functions of the attributes.
  - The IBP can also be viewed in terms of the beta process. Essentially, the beta process is to the IBP as the Dirichlet process is to the CRP.

+ Dirichlet diffusion trees, a hierarchical clustering model, where the data points cluster at different levels of granularity. I.e., there may be a few coarse-grained clusters, but these themselves might decompose into more fine-grained clusters.
+ the Pitman-Yor process, which is like the CRP, but has a more heavy-tailed distribution (in particular, a power law) over cluster sizes. I.e., you'd expect to find a few very large clusters, and a large number of smaller clusters. Power law distributions are a better fit to many real-world datasets than the exponential distributions favored by the CRP.

Sampling algorithms

From the "core topics" section, you've already learned two examples of sampling algorithms: Gibbs sampling and Metropolis-Hastings (M-H). Gibbs sampling covers a lot of the simple situations, but there are a lot of models for which you can't even compute the updates. Even for models where it is applicable, it can mix very slowly if different variables are tightly coupled. M-H is more general, but the general formulation provides little guidance about how to choose the proposals, and the proposals often need to be chosen very carefully to achieve good mixing.

Here are some more advanced MCMC algorithms which often perform much better in particular situations:

+ collapsed Gibbs sampling, where a subset of the variables are marginalized (or collapsed) out analytically, and Gibbs sampling is performed over the remaining variables. For instance, when fitting a CRP clustering model, we often marginalize out the cluster parameters and perform Gibbs sampling over the cluster assignments. This can dramatically improve the mixing, since the assignments and cluster parameters are tightly coupled.
+ Hamiltonian Monte Carlo (HMC), an instance of M-H for continuous spaces which uses

the gradient of the log probability to choose promising directions to explore. This is the algorithm that powers Stan .

+ slice sampling, an auxiliary variable method for sampling from one-dimensional distributions. Its key selling point is that the algorithm doesn't require specifying any parameters. Because of this, it is often combined with other algorithms such as HMC which would otherwise require specifying step size parameters.

+ reversible jump MCMC, a way of constructing M-H proposals between spaces of differing dimensionality. The most common use case is Bayesian model averaging.

While the majority of sampling algorithms used in practice are MCMC algorithms, sequential Monte Carlo (SMC) is another class of techniques based on approximately sampling from a sequence of related distributions.

+ The most common example is probably the particle filter, an inference algorithm typically applied to time series models. It accounts for observations one time step at a time, and at each step, the posterior over the latent state is represented with a set of particles.

+ Annealed importance sampling (AIS) is another SMC method which gradually "anneals" from an easy initial distribution (such as the prior) to an intractable target distribution (such as the posterior) by passing through a sequence of intermediate distributions. An MCMC transition is performed with respect to each of the intermediate distributions. Since mixing is generally faster near the initial distribution, this is supposed to help the sampler avoid getting stuck in local modes.
  - The algorithm computes a set of weights which can also be used to estimate the marginal likelihood. If enough intermediate distributions are used, the variance of the weights is small, and therefore they yield an accurate estimate of the marginal likelihood.

Variational inference

Variational inference is another class of approximate inference techniques based on optimization rather than sampling. The idea is to approximate the intractable posterior distribution with a tractable approximation. The parameters of the approximate distribution are chosen to minimize some measure of distance (usually KL divergence) between the approximation and the posterior.

It's hard to make any general statements about the tradeoffs between variational inference and sampling, because each of these is a broad category that includes many particular algorithms, both simple and sophisticated. However, here are some general rules of thumb:

+ Variational inference algorithms involve different implementation challenges from sampling algorithms:
  - They are harder, in that they may require lengthy mathematical derivations to determine the update rules.
  - However, once implemented, variational Bayes can be easier to test, because one can employ the standard checks for optimization code (gradient checking, local optimum tests, etc.)
  - Also, most variational inference algorithms converge to (local) optima, which eliminates the need to check convergence diagnostics.
+ The output of most variational inference algorithms is a distribution, rather than samples.
  - To answer many queries, such as the expectation or variance of a model parameter, one can simply check the variational distribution. With sampling methods, by contrast, one often needs to collect large numbers of samples, which can be expensive.
  - However, with variational methods, the accuracy of the approximation is limited by the expressiveness of the approximating class, and it's not always obvious how

different the approximating distribution is from the posterior. By contrast, if you run a sampling algorithm long enough, eventually you will get accurate results.

Here are some important examples of variational inference algorithms:

+ variational Bayes, the application of variational inference to Bayesian models where the posterior distribution over parameters cannot be represented exactly. If the model also includes latent variables, then variational Bayes EM can be used.
+ the mean field approximation, where the approximating distribution has a particularly simple form: all of the variables are assumed to be independent.
    - Mean field can also be viewed in terms of convex duality, which leads to different generalizations from the usual interpretation.
+ expectation propagation, an approximation to loopy belief propagation. It sends approximate messages which represent only the expectations of certain sufficient statistics of the relevant variables.

And here are some canonical examples where variational inference techniques are applied. While you're unlikely to use these particular models, they provide a guide for how variational techniques can be applied to Bayesian models more generally:

+ linear regression
+ logistic regression
+ mixture of Gaussians
+ exponential family models

Belief propagation

Belief propagation is another family of inference algorithms intended for graphical models such as Bayes nets and Markov random fields (MRFs). The variables in the model "pass

messages" to each other which summarize information about the joint distribution over other variables. There are two general forms of belief propagation:

+ When applied to tree-structured graphical models, BP performs exact posterior inference. There are two particular forms:
  - the sum-product algorithm, which computes the marginal distribution of each individual variable (and also over all pairs of neighboring variables).
  - the max-product algorithm, which computes the most likely joint assignment to all of the variables
+ It's also possible to apply the same message passing rules in a graph which isn't tree-structured. This doesn't give exact results, and in fact lacks even basic guarantees such as convergence to a fixed point, but often it works pretty well in practice. This is often called loopy belief propagation to distinguish it from the tree-structured versions, but confusingly, some research communities simply refer to this as "belief propagation."
  - Loopy BP can be interpreted as a variational inference algorithm.

The junction tree algorithm gives a way of applying exact BP to non-tree-structured graphs by defining coarser-grained "super-variables" with respect to which the graph is tree-structured.

The most common special case of BP on trees is the forward-backward algorithm for HMMs. Kalman smoothing is also a special case of the forward-backward algorithm, and therefore of BP as well.

BP is widely used in computer vision and information theory, where the inference problems tend to have a regular structure. In Bayesian machine learning, BP isn't used very often on its own, but it can be a powerful component in the context of a variational or sampling-based algorithm.

Theory

Finally, here are some theoretical issues involved in Bayesian methods.

+ Defining a Bayesian model requires choosing priors for the parameters. If we don't have strong prior beliefs about the parameters, we may want to choose uninformative priors. One common choice is the Jeffreys prior.
+ How much data do you need to accurately estimate the parameters of your model? The asymptotics of maximum likelihood provide a lot of insight into this question, since for finite models, the posterior distribution has similar asymptotic behavior to the distribution of maximum likelihood estimates.